STM32基础 - I2C通信

课程目标

1. 熟悉I2C通信协议。
2. 熟悉MPU6050，理解其加速度计和陀螺仪工作原理。
3. 熟悉STM32的I2C外设工作过程，能够利用I2C协议控制MPU6050。

课程试验

1. 利用STM32完成软件I2C读取MPU6050试验
2. 利用STM32完成硬件I2C读取MPU6050试验

课堂引入

与USART一样，I2C也是通信协议的一种。I2C用于连接微控制器和外围设备之间的主从通信，多在小数据量场合使用。I2C传输距离较短、传输速度在标准模式下可达100kbps，快速模式下可达400kbps。

和串口的对比

同/异步：

- 串口通讯是异步，没有时钟线，串口通讯依赖硬件电路（软件不容易模拟）
- I2C是同步协议，有时钟线SCL（控制数据发送接收时机），由于有时钟线，所以很容易用软件模拟，也可以有硬件支持。
- 双工： 串口是全双工（Rx,Tx），I2C是半双工(SDA)
- 设备： 串口是点对点；I2C是1主对多从，多主多从

I2C优势：

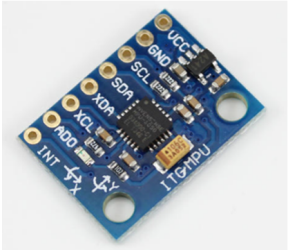
- 在硬件不足的情况下，可以用软件模拟
- 支持1主多从
- 时钟同步的。（不受中断影响）

授课进程

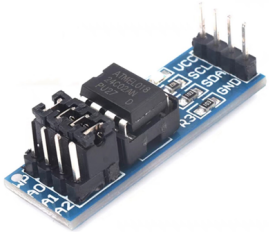
一、I2C通信

1、基本介绍

I2C（Inter IC Bus）是由Philips公司开发的一种通用数据总线，目前应用非常广泛。比如下图左一是后需要讲的MPU6050姿态测量模块，左二是一直有用到的OLED模块，图三是AT24C02存储器模块，图四则是DS3231实时时钟模块，它们都是支持I2C通信协议的。对于开发者来说在不同的硬件上使用了相同的通信协议，就意味着学会了其中一个硬件，再学其他的硬件就很容易了。I2C的标志性引脚就是两根通信线：一是SCL，全称Serial Clock串行时钟线，二是SDA，全称Serial data串行数据线。SCL时钟线使用同步的时序，可以降低对硬件的依赖，同时同步的时序稳定性也会比异步的时序更高。只有一根SDA数据线，将全双工通信变为了半双工通信，一根线兼具发送和接收，最大化的利用了资源。I2C协议是同步半双工的协议，也是一个带数据应答的协议。支持总线挂载多设备，分为两种模型：一种是一主多从模型。一主多从就是单片机作为主机主导I2C总线的运行，挂载在I2C总线的所有外部模块都是从机，从机只有被主机点名之后，才能控制I2C总线，不能在未经允许的情况下去碰I2C总线，从而防止冲突产生。绝大多数场景下使用I2C都是采用一主多从的模型。另一种是多主多从模型。多主多从就是在总线上的任何一个模块都可以主动跳出来要求做主机，当已经有一个模块成为主机，并且它还在工作时又有另外一个模块申请成为主机时就会发生总线冲突。发生总线冲突后，I2C协议会进行仲裁，仲裁胜利的一方将取得总线的控制权，失败的一方自动变回从机。当然由于时钟线也是由主机控制的，所以在多主多从模型下，还需要进行时钟同步，这种多主机情况下的协议还是比较复杂的。





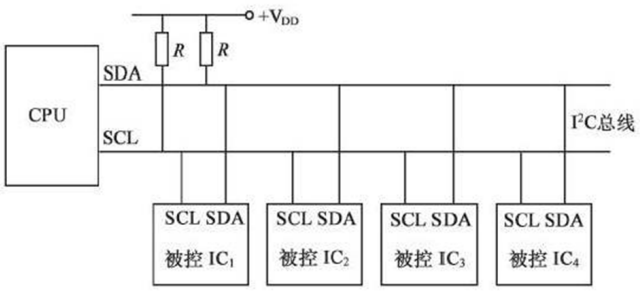


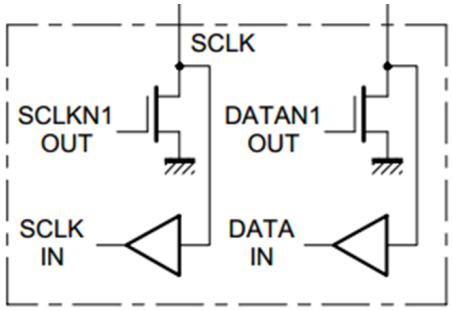


2、硬件电路

下边左图是I2C典型的一主多从模型：

左边CPU是单片机作为总线的主机，主机的权利很大，包括对SCL线的完全控制，任何时候都是主机完全掌控SCL线。在空闲状态下，主机可以主动发起对SDA的控制。只有在从机发送数据和从机应答的时候，主机才会转交SDA的控制权给从机。图中一系列被控IC都是挂载在I2C总线上的从机，这些从机可以是姿态传感器、OLED、存储器、时钟模块等等。从机的权利比较小，对于SCL时钟线，在任何时刻都只能被动的读取。从机不允许控制SCL线。对于SDA数据线，从机不允许主动发起对SDA的控制，只有在主机发送读取从机的命令后，或者从机应答的时候，从机才能短暂的获取SDA的控制权，这是一主多从模型中协议的规定。在接线上所有I2C设备的SCL连在一起，SDA连在一起。对于SCL因为是一主多从模型下，主机拥有SCL的绝对控制权，所以主机的SCL可以配置成推挽输出，所有从机的SCL都配置成浮空输入或者上拉输入，此时数据流向是主机发送，所有从机接收，这里没有问题。但是SDA就比较麻烦了，由于I2C是半双工协议，所以主机和从机都是发送的时候是输出，接受的时候是输入。如果能协调好输入输出的切换时机，是没有问题的。但是有很大的可能会出现总线时序没有协调好，两个设备引脚同时处于输出状态，而恰好一个在输出高电平，一个在输出低电平，拿这个状态就会导致两个设备引脚检出现电源短路的情况，这种状态是必须想办法避免的。因此I2C禁止所有设备输出强上拉的高电平，采用外置弱上拉电阻加开漏输出的电路结构，因此图中SCL和SDA各添加了一个上拉电阻，阻值一般为4.7千欧左右。



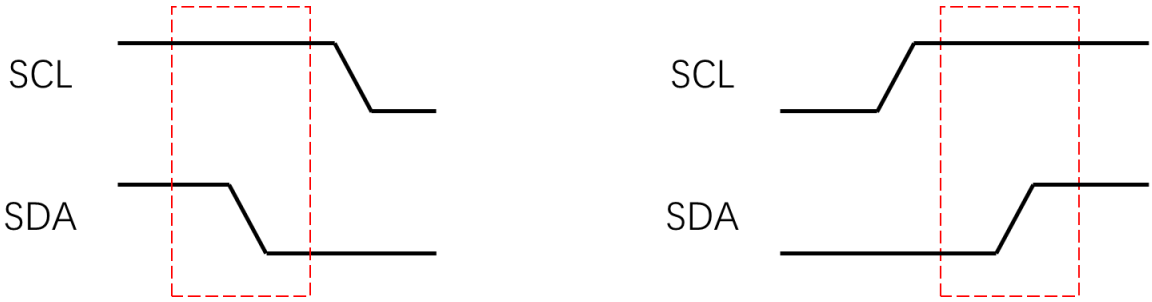


上边右图左边是SCLK的结构，图中SCLK就是SCL。右边是SDA的结构，图中DATA就是SDA。左图中所有的CPU与被控IC的SCL、SDA引脚的内部结构都和这个图中展示的是一样的。先看SCLK这一路，首先引脚的信号进来，都可以通过一个数据缓冲器或者是斯密特触发器进输入，因为输入对电路没有任何影响。所以任何设备在任何时刻都是可以输入的，但是在输出的部分采用的是开漏输出的配置。开漏输出没有了强上拉的开关管，在输出低电平时，开关管导通是强下拉；输出高电平时，开关管断开，由于没有了强上拉开关管，此时引脚处于浮空的状态，这就是开漏输出。这样所有的设备都只能输出低电平，而不能输出高电平。为了避免高电平造成的引脚浮空，就需要在总线外面的SCL和SDA各外置一个上拉电阻，通过一个电阻拉到高电平，这就是一个弱上拉的高电平。这样第一完全杜绝了电源短路现象，保证电路的安全。第二避免了引脚模式的频繁切换，开漏加弱上拉的模式同时兼具了输入和输出的功能。因为开漏模式下，输出高电平就相当于断开硬件，所以在输入之前可以直接输出高电平，不需要再切换成输入模式。第三就是这个模式会有个线与的现象，就是只要有任意一个或多个设备输出了低电平，总线就处于低电平，只有所有的设备都输出高电平，总线才处于高电平。I2C可以利用这个电路特征执行多主机模式下的时钟同步和总线仲裁，所以这里SCL虽然在一主多从模式下可以用推挽输出，但是它仍然采用了开漏加上拉输出的模式
因为在多主机模式下会利用到这个特征。

3、I2C时序基本单元

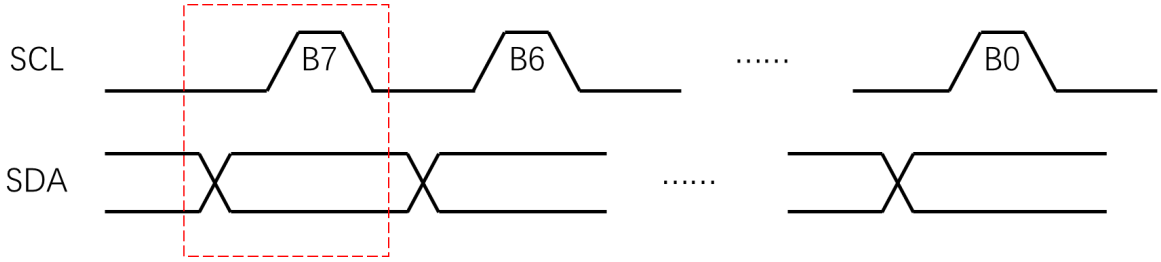
起始条件：SCL高电平期间，SDA从高电平切换到低电平

终止条件：SCL高电平期间，SDA从低电平切换到高电平



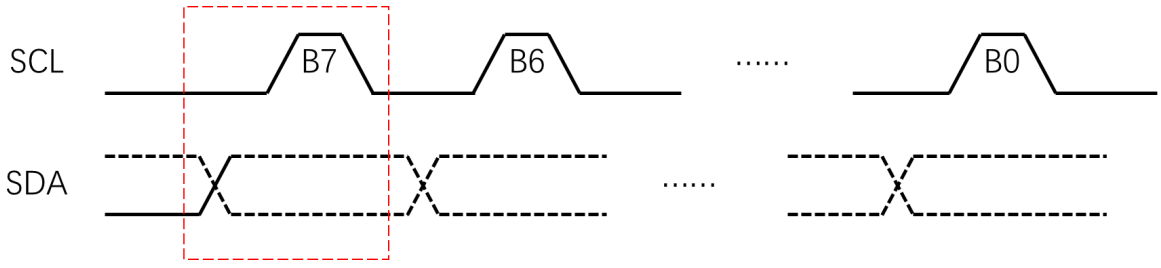
（1）发送一个字节

SCL低电平期间，主机将数据位依次放到SDA线上（高位先行），然后释放SCL，从机将在SCL高电平期间读取数据位，所以SCL高电平期间SDA不允许有数据变化，依次循环上述过程8次，即可发送一个字节。



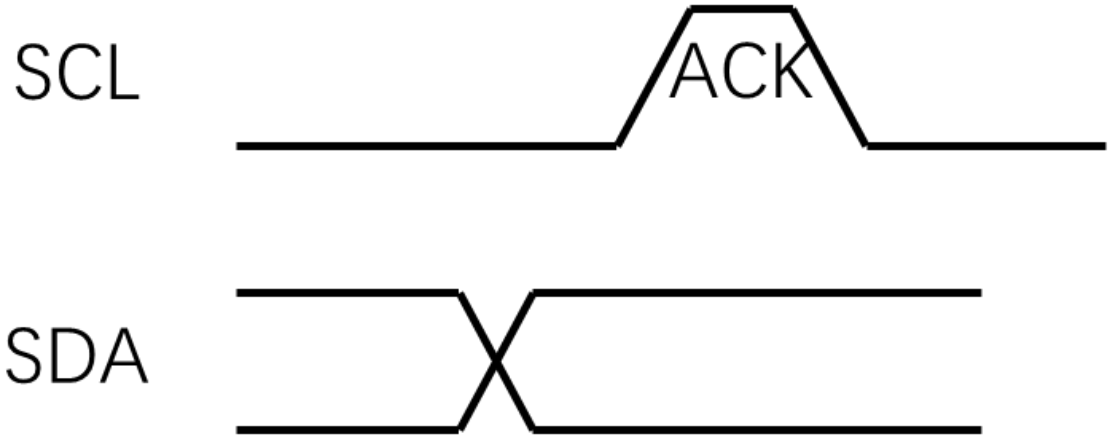
（2）接收一个字节

SCL低电平期间，从机将数据位依次放到SDA线上（高位先行），然后释放SCL，主机将在SCL高电平期间读取数据位，所以SCL高电平期间SDA不允许有数据变化，依次循环上述过程8次，即可接收一个字节（主机在接收之前，需要释放SDA）。



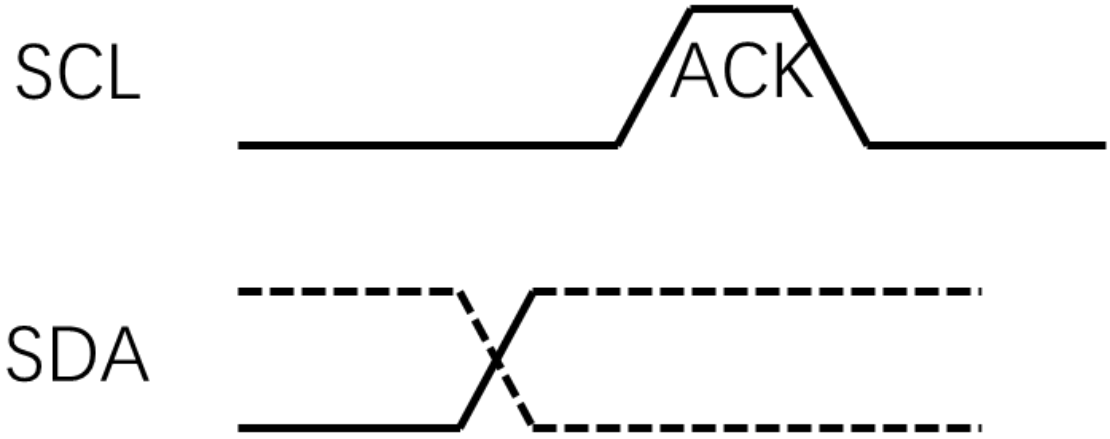
（3）发送应答

主机在接收完一个字节之后，在下一个时钟发送一位数据，数据0表示应答，数据1表示非应答。



（4）接收应答

主机在发送完一个字节之后，在下一个时钟接收一位数据，判断从机是否应答，数据0表示应答，数据1表示非应答（主机在接收之前，需要释放SDA）。



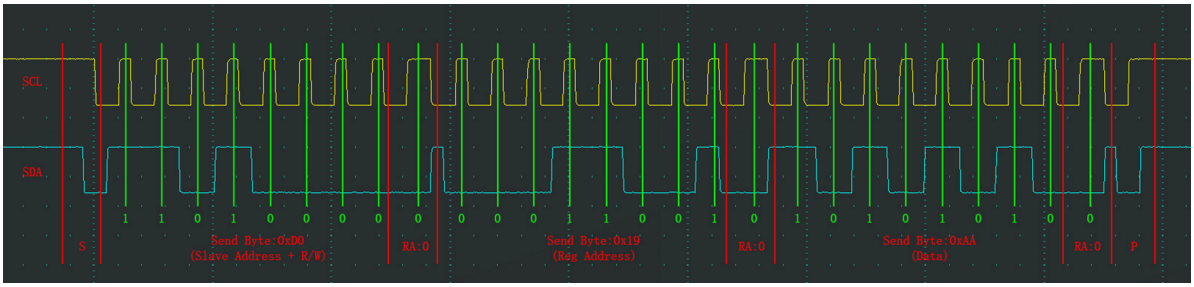
4、I2C时序

组成一个数据帧，I2C的完整时序主要有指定地址写、当前地址读以及指定地址读这三种。

在一主多从的I2C模型下，主机可以访问总线上的任何一个设备，那如何发送指令来确定访问的是哪个设备呢？这需要首先把每个从机都确定一个唯一的设备地址，从机的设备地址就相当于每个设备的名字。在同一条I2C总线里，挂载的每个设备地址必须不一样，否则主机叫一个地址，有多个设备都响应，就乱套了。从机设备地址在I2C协议标准里分为7位地址和10位地址，这里重点讲7位地址的模式。因为7位地址比较简单，而且应用范围最广。在每个I2C设备出厂时，厂商都会为它分配一个7位的地址，一般不同型号的芯片地址都是不同的，而相同型号的芯片地址都是一样的。如果有相同的芯片挂载在同一条总线上，就需要用到地址中的可变部分，一般器件地址的最后几位是可以在电路中改变的。比如MPU6050地址的最后一位，就可以由这个板子上的AD0引脚确定。这个引脚接低电平，它的地址就是1101000，引脚接高电平，那它的地址就是1101001。一般I2C的从机设备地址高位都是由厂商确定的，低位可以由引脚来灵活切换，这样即使相同型号的芯片挂载在同一个总线上，也可以通过切换地址低位的方式，保证每个设备的地址都不一样。

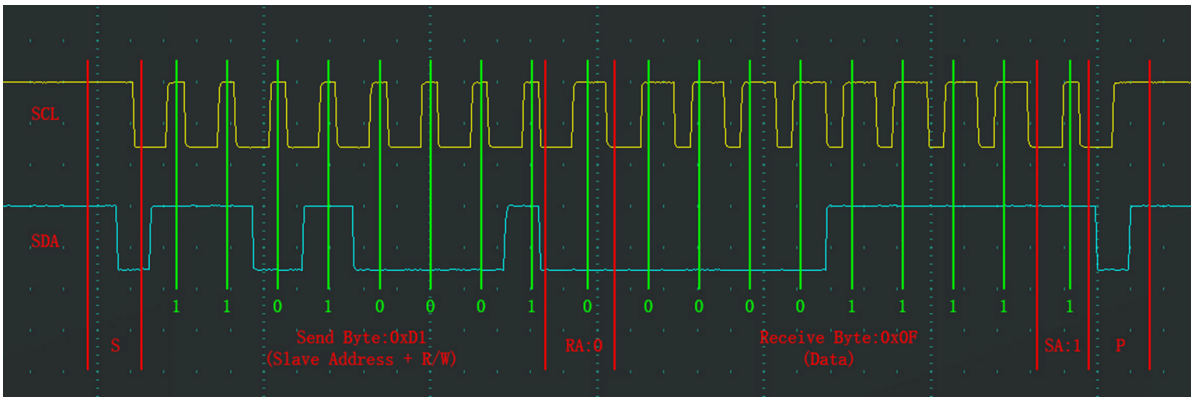
（1）指定地址写

对于指定设备（Slave Address），在指定地址（Reg Address）下，写入指定数据（Data）



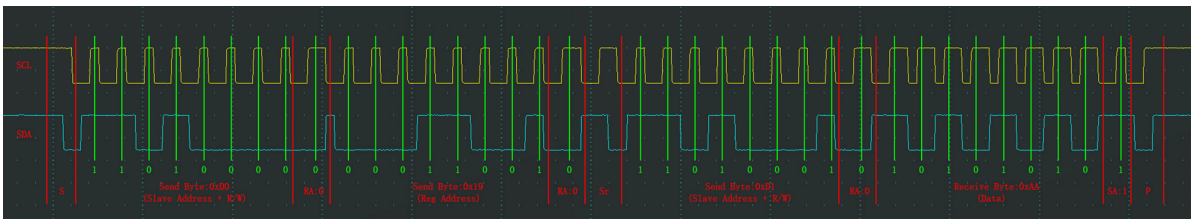
（2）当前地址读

对于指定设备（Slave Address），在当前地址指针指示的地址下，读取从机数据（Data）



（3）指定地址读

对于指定设备（Slave Address），在指定地址（Reg Address）下，读取从机数据（Data）



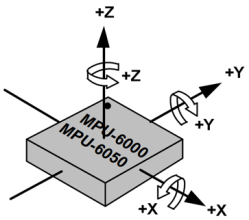
二、MPU6050

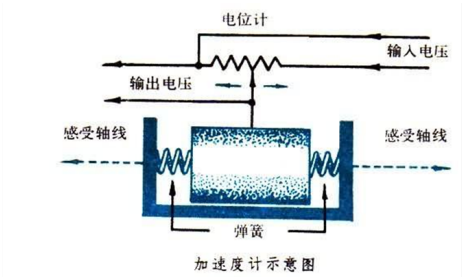
1、简介

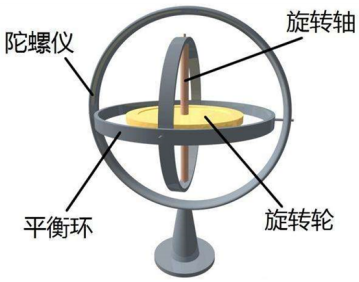
MPU6050是一个6轴姿态传感器，可以测量芯片自身X、Y、Z轴的加速度、角速度参数，通过数据融合，可进一步得到姿态角，常应用于平衡车、飞行器等需要检测自身姿态的场景。

3轴加速度计（Accelerometer）：测量X、Y、Z轴的加速度。

3轴陀螺仪传感器（Gyroscope）：测量X、Y、Z轴的角速度。







MPU6050内部的六个轴传感器包含了一个三轴的加速度计传感器和一个三轴的陀螺仪传感器。

三轴加速度记的英文全称是Accelerometer。一般简称Accel或者Acc，或者A。加速度计的作用就是测量X、Y、Z轴的加速度。这里X、Y、Z轴就是一个三维的坐标系。其中横向的轴定义为X轴，纵向的轴定义为Y轴，垂直于芯片的轴定义为Z轴。在芯片内部每个轴都分别布置了一个加速度计。上面中间的图就是一个加速度计的结构图。其中水平的虚线是感应轴线，中间是一个具有一定质量可以左右滑动的小滑

块，然后左右各有一个弹簧顶着，当把这个东西拿在手上来回晃，中间的小滑块就会左右移动，去压缩或者拉伸两边的弹簧。当滑块移动时，就会带动上面的电位器滑动，这个电位器其实就是一个分压电阻，然后通过测量电位器输出的电压，就能得到小滑块所受的加速度值了。所以加速度计实际上就是一个弹簧测力计。在这个芯片里面，X、Y、Z三个轴分别都有这样一个测力计。举例来说，可以想象在这个芯片里面有六个侧立的秤组成一个正方体。然后正方体内部放一个大小正好的单位质量小球，这个小球压在哪个称面上，就会产生对应一个轴的数据输出。比如小球压在下面，这个面就是Z轴的正值，小球压上面，这个面就是Z轴的负值。对向两个面一组，一面为正值一面为负值，那正方体六个面所测的力就是三个轴的加速度值，这是加速度计的一个直观理解。比如把芯片静置水平放在地球上，就只有底面测力计受到小球的压力，所以此时数据输出就是X、Y轴输出为0，Z轴输出一个g的加速度值。如果此时芯片正处于自由落体状态，那就是所有面都不受力，此时数据输出就是X、Y、Z轴都为0。如果此时芯片向左倾斜放置，就是底面和左面都受力，这时求一个三角函数，就能得到向左的倾角θ了。不过这个倾角只有芯片静置的时候才是正确的，因为加速度分为重力加速度和运动加速度，如果此时芯片运动起来了，这个三角函数求得的倾角就会受运动加速度的影响。举个例子，比如人坐在汽车里，现在汽车突然向前加速，就会感觉椅子底面和靠背都受力。这时如果用三角函数求角度，得到的结果就是车停在一个斜坡上。停在斜坡上就是椅子底面和靠背都受力，但实际上车的状态却是水平向前加速，所以仅使用加速度计求角度，只能在物体静止的时候使用，当物体运动起来了，这个角度就会受运动加速度的影响而变得不准确。这就是加速度计的测量原理和特征，**总结一下，就是加速度计具有静态稳定性，不具有动态稳定性。**

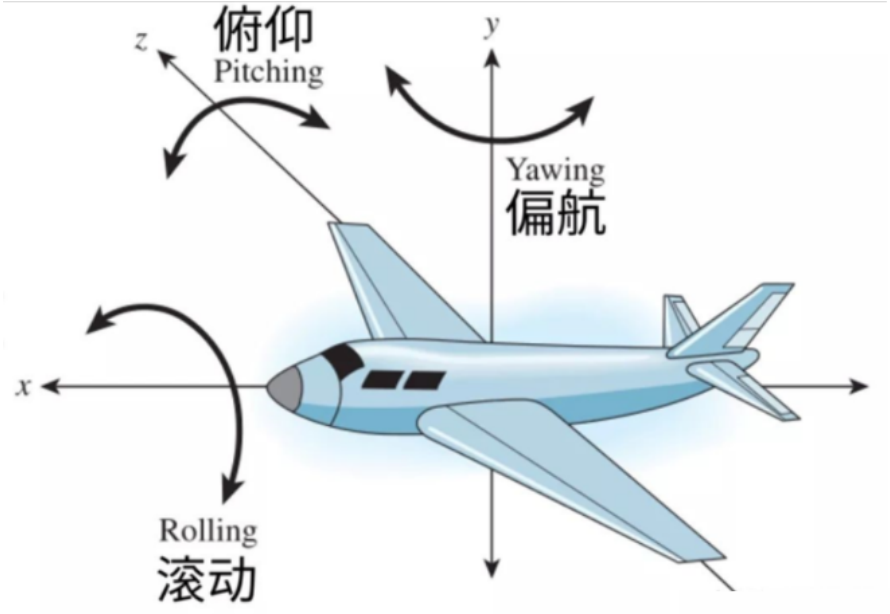
三轴陀螺仪传感器的英文全称是Gyroscope。一般简称Gyro或者G。陀螺仪传感器的作用是测量X、Y、Z轴的角速度。上面最右边的图就是陀螺仪的机械模型。中间是一个有一定质量的旋转轮，外面是三个轴的平衡环。当中间这个旋转轮高速旋转时，根据角动量守恒的原理，这个旋转轮具有保持它原有角动量的趋势，这个趋势可以保持旋转轴方向不变。当外部物体的方向转动时，内部的旋转轴方向并不会转动，这就会在平衡环连接处产生角度偏差。如果在连接处放一个旋转的电位器，测量电位器的电压，就能得到旋转的角度了。照此分析陀螺仪应该是可以直接得到角度的，但是MPU6050的陀螺仪并不能直接测量角度，它芯片内部的陀螺仪测量的实际上是角速度，而不是角度。陀螺仪测量X、Y、Z轴的角速度值分别表示了此时芯片绕X轴、绕Y轴和绕Z轴旋转的角速度。举个例子，可以把测量角速度的陀螺仪想象成是游乐园的旋转飞椅，中间的轴转得越快，这个椅子就飞得离中间轴越远，最终测量对向两个椅子飞起来的距离，或者飞起来的夹角，就能得到中间轴的角速度了，这就是测量角速度的陀螺仪的工作原理。如果还想通过角速度得到角度，只需要对角速度进行积分即可，角速度积分就是角度。要注意的是这个角速度积分得到的角度，也有局限性。就是当物体静止时，角速度值会因为噪声无法完全归零。然后经过积分的不断累积，这个小噪声，就会导致计算出来的角度产生缓慢的漂移，也就是角速度积分得到的角度经不起时间的考验。不过这个角度，无论是运动还是静止都是没问题的，它不会受物体运动的影响**。总结一下，就是陀螺仪具有动态稳定性，不具有静态稳定性。这个陀螺仪是动态稳定，静态不稳定的。**

这两种传感器的特性正好互补，所以取长补短，进行一下互补滤波，就能融合得到静态和动态都稳定的姿态角，这就是姿态解算的大体思路。当然，实际的姿态解算就更加复杂些了，这里暂时就不做深入了，我们还是把学习重点放在I2C对MPU6050的操控上来。

使用加速度和角速度进行互补滤波、卡尔曼滤波或DMP姿态融合可以完成姿态解,得到欧拉角。

欧拉角：

俯仰角（Pitch） 横滚角(Roll) 偏航角(yaw)



2、MPU6050参数

16位ADC采集传感器的模拟信号，量化范围：-32768~32767。

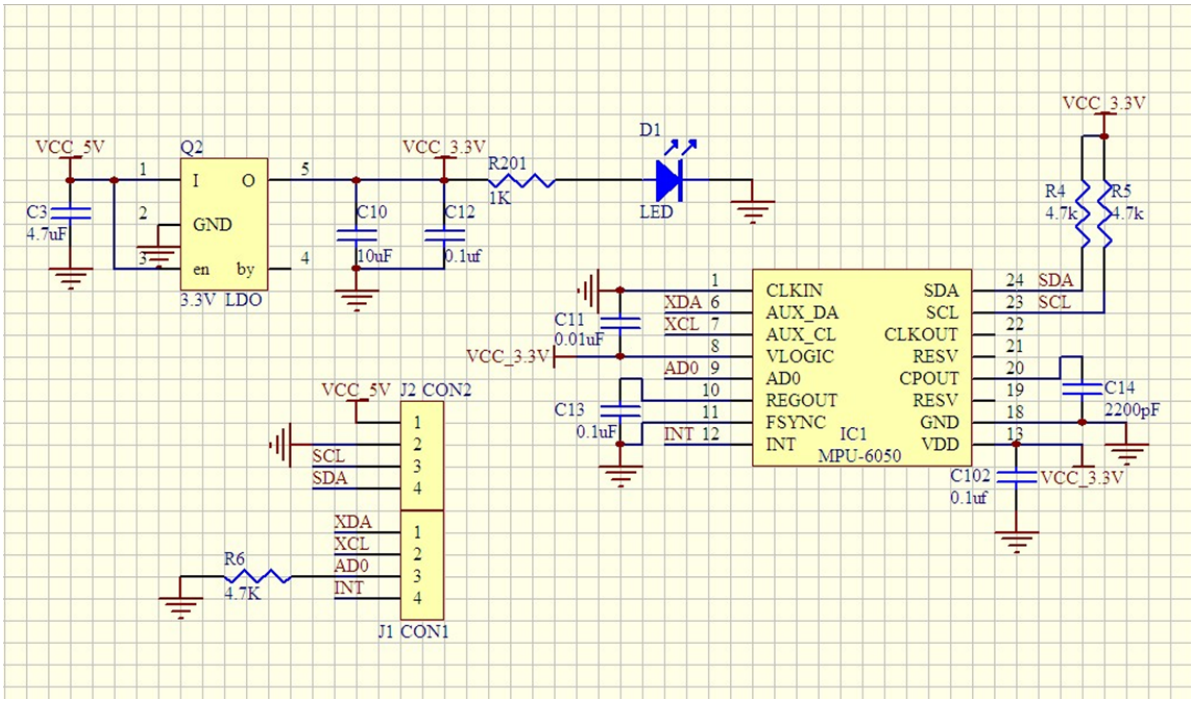
加速度计满量程选择：±2、±4、±8、±16（g）。

陀螺仪满量程选择： ±250、±500、±1000、±2000（°/sec）。

具有可配置的数字低通滤波器，可配置的时钟源，可配置的采样分频。

其I2C从机地址固定为11010000（D0融入读写位的，如果不融入读写位，那地址应该是110 1000 地址就是0x68），但是可以通过AD0引脚接入高低电平来修改低位地址。当AD0引脚接入低电平时，地址为1101000，当AD0引脚接入高电平时，地址为1101001。

3、硬件电路

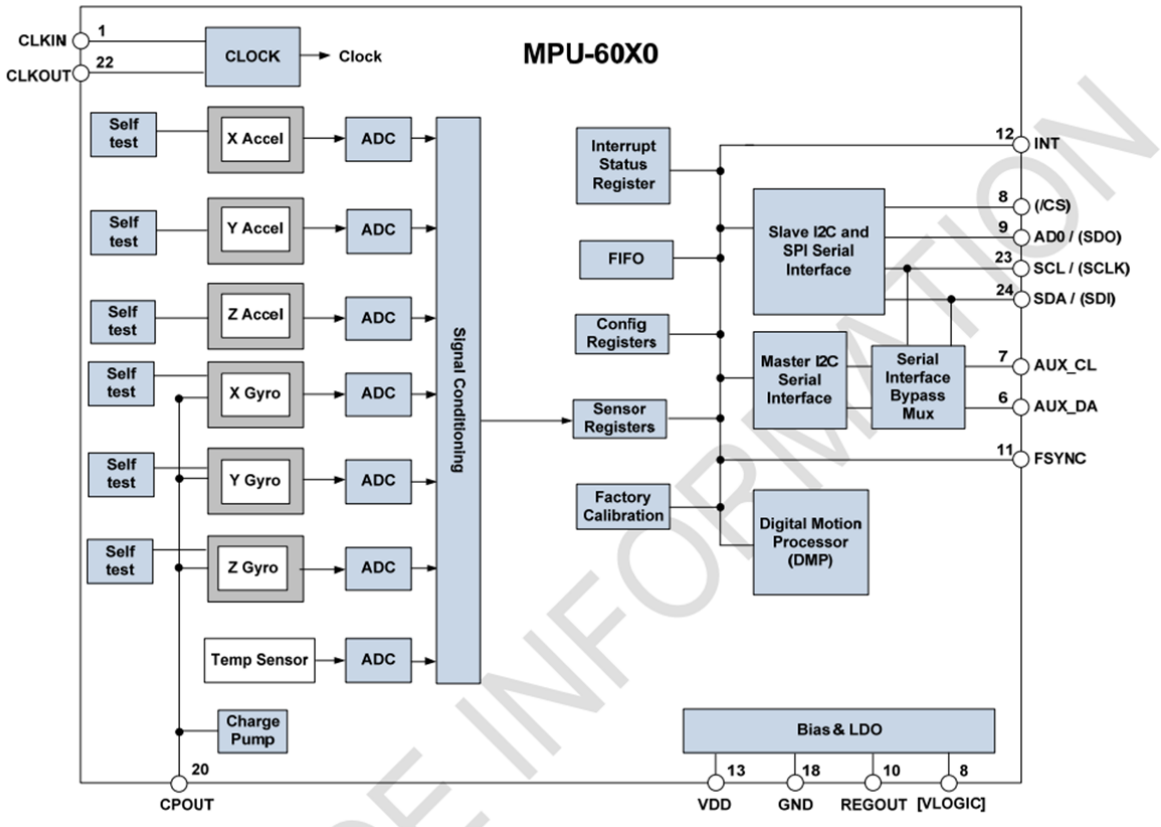


MPU6050模块的原理图，其中右边是MPU6050的芯片，左下角是一个8针的排针，左上角是一个LDO，低压差线性稳压器。先看右边MPU6050的芯片，芯片本身的引脚是非常多的，包括时钟、I2C通信引脚、供电、帧同步等等。不过这里有很多引脚都用不到，还有些引脚这个芯片最小系统里的固定连接。最小系统一般手册里都会有，抄过来就行了。左下角排针上引出来的引脚有VCC和GND，这两个引脚是电源供电，还有SCL和SDA，这两个引脚是I2C通信的引脚。通过最右上角的电路可以看出SCL和

SDA模块已经内置了两个4.7 k的上拉电阻了，所以在接线的时候，直接把SCL和SDA接在GPIO口就行了，不需要再在外面另外接上拉电阻了。咱看排针下面有XCL和XDA，这两个是芯片里面的主机I2C通信引脚，设计这两个引脚是为了扩展芯片功能。MPU6050是一个六轴姿态传感器，只有加速度计和陀螺仪的六个轴融合出来的姿态角，这是有缺陷的。缺少校正感知方向的磁力计和感知高度差的气压计传感器。如果应用场景需要这些，可以利用这个XCL和XDA引脚，扩展外接磁力计或者气压计。当接上磁力计或气压计之后，MPU6050的主机接口可以直接访问这些扩展芯片的数据，把这些扩展芯片的数据读取到MPU6050里面。在MPU6050里面会有DMP单元进行数据融合和姿态解算。下面的AD0引脚是用来调整从机地址的最低位的。接低电平的话，7位从机地址就是1001000，接高电平的话，七位从机地址就是1001001。图中可以看到有一个电阻默认弱下拉到低电平了。排针的最后一个引脚是INT，也就是中断输出引脚。可以配置芯片内部的一些事件，来触发中断引脚的输出，比如数据准备好了，I2C主机错误等。另外，芯片内部还内置了一些实用的小功能。比如自由落体检测、运动检测、零运动检测等，这些信号都可以触发INT引脚产生电平跳变，需要的话可以进行中断信号的配置。但如果不需要，可以不配置这个引脚。最后看左上角的LDO，这部分是供电的逻辑。在手册里可以查到MPU6050芯片的VDD供电是2.375-3.46伏，属于3.3伏供电的设备，不能直接接5伏。所以为了扩大供电范围，这个模块的设计者就加了个3.3伏的稳压器，输入端电压VCC_5V，可以在3.3伏到5伏之间，然后经过3.3伏的稳压器，输出稳定的3.3伏电压给芯片端供电。此外在3.3伏端接了一个电源指示灯，只要有电，电源指示灯就会亮。以上就是MPU6050模块的电路原理了。

引脚	功能
VCC、GND	电源
SCL、SDA	I2C通信引脚
XCL、XDA	主机I2C通信引脚
AD0	从机地址最低位
INT	中断信号输出

4、MPU6050框图



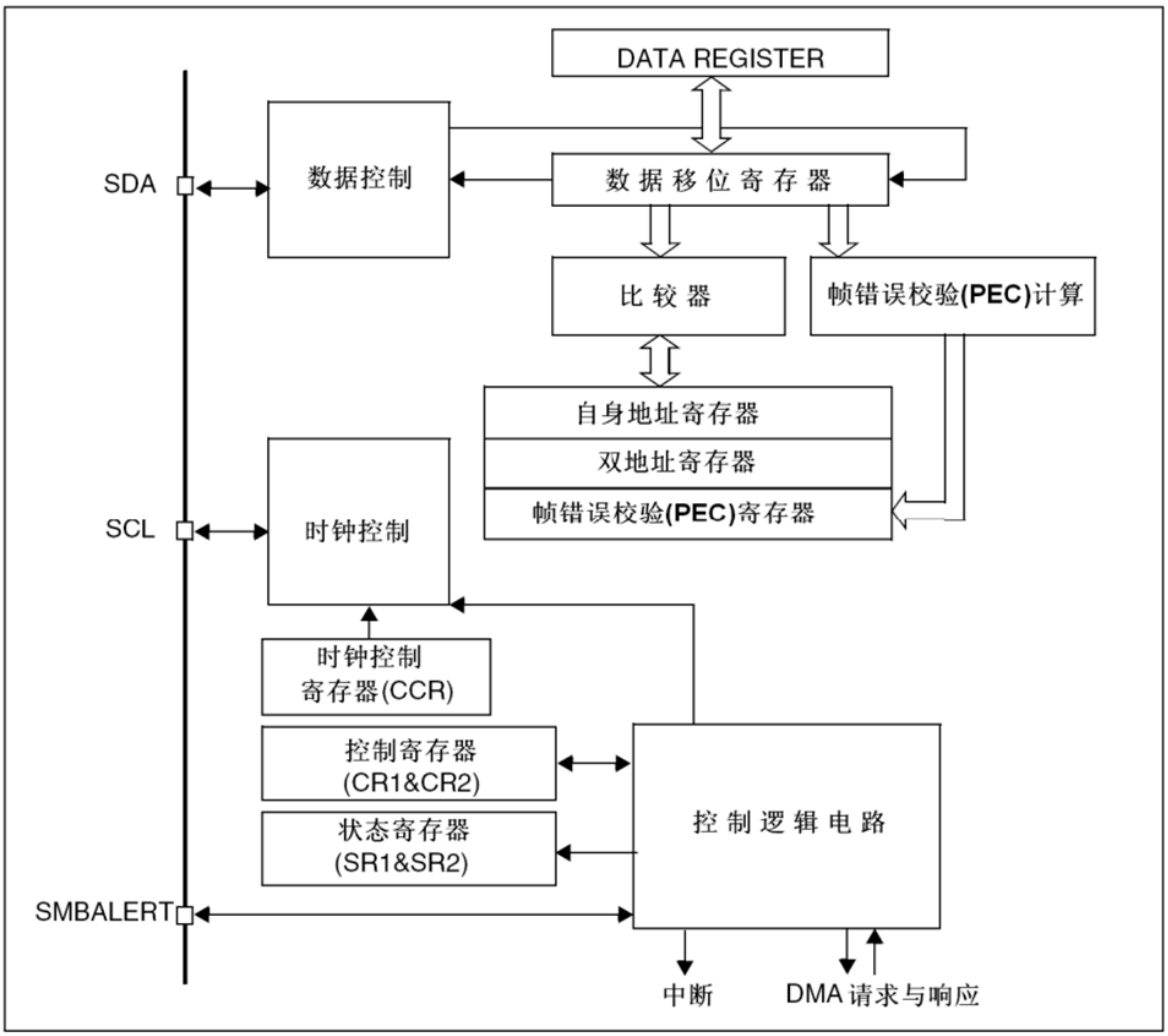
三、I2C硬件外设

1、基本介绍

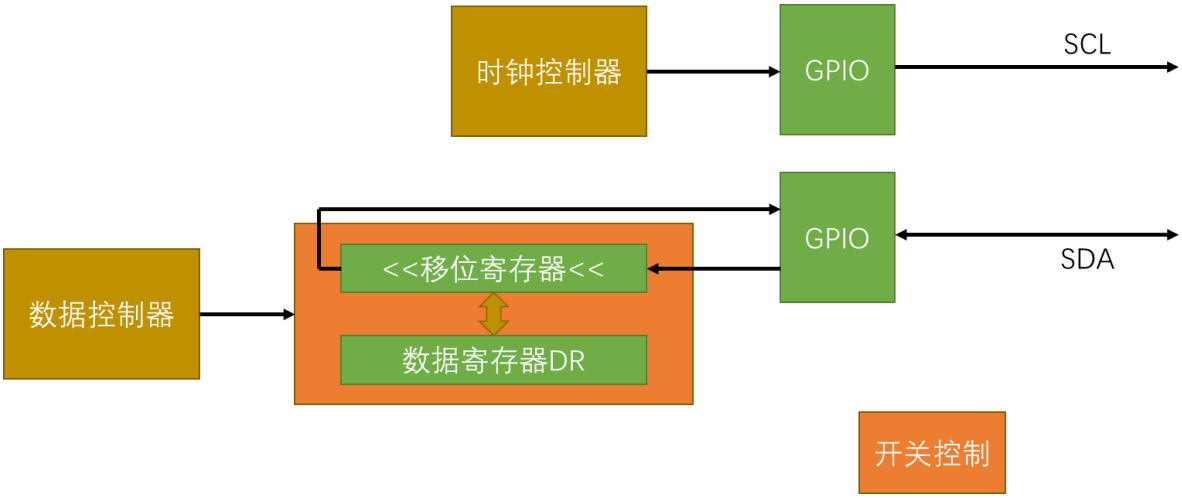
STM32内部集成了硬件I2C收发电路，可以由硬件自动执行时钟生成、起始终止条件生成、应答位收发、数据收发等功能，减轻CPU的负担，节省软件资源。它支持多主机模型，支持7位/10位地址模式，支持不同的通讯速度，标准速度(高达100 kHz)，快速(高达400 kHz)，支持DMA，兼容SMBus协议。对于STM32F103C8T6芯片包含的硬件I2C资源有I2C1和I2C2这两组。

2、I2C框图

图242　I^2C的功能框图



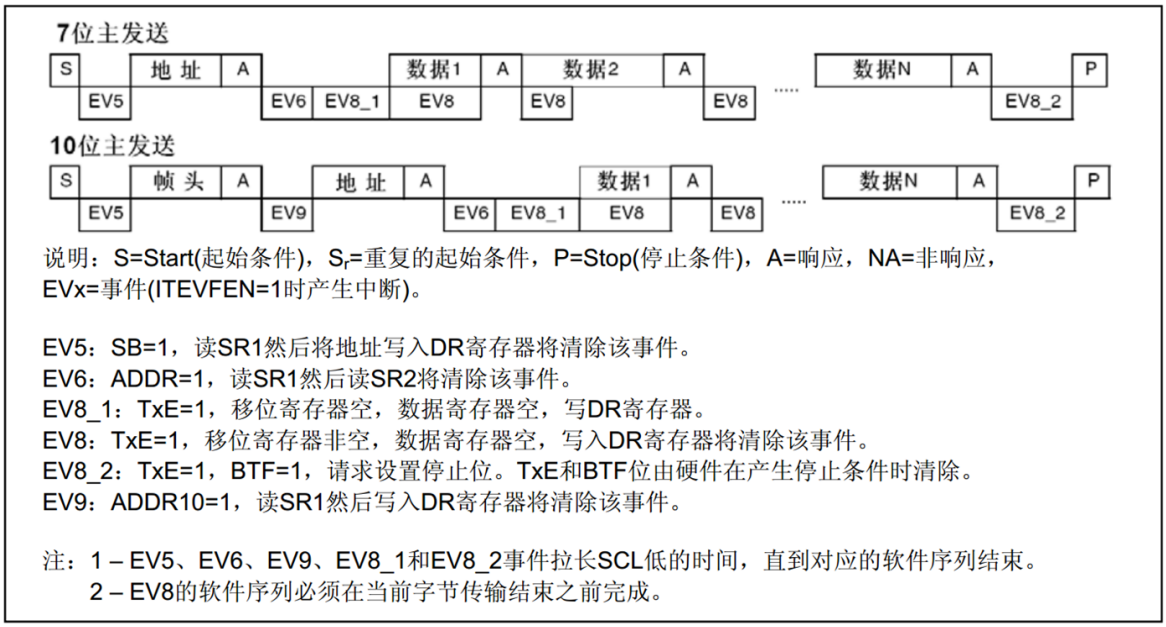
3、I2C基本结构



这是I2C基本结构图，图中把没有用到的东西简化掉了，仅保留了必须使用的部分。I2C工作时移位寄存器和数据寄存器DR的配合是通信的核心部分。因为I2C是高位先行，所以移位寄存器是向左移位。在发送的时候最高位先移出去，然后是次高位等等。一个SCL时钟移位1次，这样移位8次就可以把一个字节由高位到低位，依次放到SDA线上了。在接收的时候，数据通过GPIO口从右边依次移进来，最终移8次，一个字节就接收完成。在GPIO口使用硬件I2C的时候，对应的两个GPIO口都要配置成复用开漏输出的模式，复用就是GPIO口的状态是交由片上外设来控制的，开漏输出则是I2C协议要求的端口配置。此外，GPIO口也是可以进行输入的，在多主机的模型中时钟线也是会进行输入的。再看SDA的部分，输出数据通过GPIO输出到端口，输出数据也是通过GPIO输入的。这里GPIO引脚复用了输入和输出，它们都有效，具体大家可以参考前面学习过的GPIO内部框图再回顾一下。数据控制器主要负责管理数据的传输过程，包括发送和接收数据。在发送数据时，数据控制器会将要发送的数据写入到数据寄存器中，然后通过SDA信号将数据发送出去。数据控制器还会控制时钟线（SCL）的时钟信号，以控制数据传输的速度和方向。在接收数据时，数据控制器会通过SDA信号接收从设备发送的数据，然后将其写入到数据寄存器中。同时数据控制器还会控制时钟线（SCL）的时钟信号，以同步数据的传输。此外，数据控制器还需要处理应答信号和停止信号等控制信号，以确保数据传输的正确性和可靠性。最后图上的开关控制就是I2C_CMD，使能外设，这样外设就能正常工作了。

4、主机发送

图245　主发送器传送序列图



说明：S=Start(起始条件)，S_r=重复的起始条件，P=Stop(停止条件)，A=响应，NA=非响应，EVx=事件(ITEVFEN=1时产生中断)。

EV5：SB=1，读SR1然后将地址写入DR寄存器将清除该事件。
EV6：ADDR=1，读SR1然后读SR2将清除该事件。
EV8_1：TxE=1，移位寄存器空，数据寄存器空，写DR寄存器。
EV8：TxE=1，移位寄存器非空，数据寄存器空，写入DR寄存器将清除该事件。
EV8_2：TxE=1，BTF=1，请求设置停止位。TxE和BTF位由硬件在产生停止条件时清除。
EV9：ADDR10=1，读SR1然后写入DR寄存器将清除该事件。

注：1 – EV5、EV6、EV9、EV8_1和EV8_2事件拉长SCL低的时间，直到对应的软件序列结束。
2 – EV8的软件序列必须在当前字节传输结束之前完成。

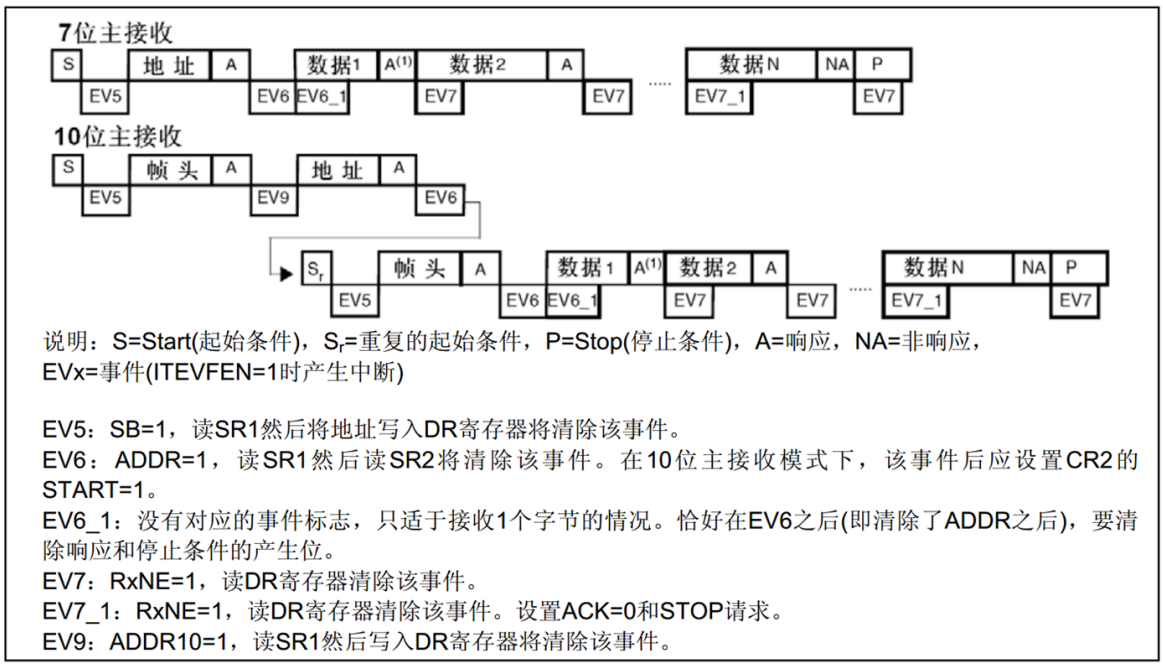
当STM32想要执行指定地址写的时候，就要按照这个主发送器传送序列图来进行。包含有7位地址的主发送和10位地址的主发送。它们的区别就是7位地址，起始条件后的一个字节是寻址。10位地址，起始条件后的两个字节都是寻址，其中前一个字节是帧头，内容是5位的标志位，11110+2位地址+1位读写位，然后后一个字节，内容就是纯粹的8位地址了，两个字节加一起构成10位的寻址，这就是10位地址的寻址模式。10位地址了解即可，我们主要关注7位地址寻址就行了。7位主发送时序流程是起始，从机地址，应答，后面是数据1，应答，数据2，应答等等，这样来表示的，最后是P，停止。因为I2C协议只规定了起始之后，必须是寻址，至于后面数据的用途，并没有明确的规定。这些数据可以由各个芯片厂商自己来规定，比如MPU6050规定，就是寻址之后，数据1为指定寄存器地址，数据2为指定寄存器地址下的数据，之后的数据N，就是从指定寄存器地址开始，依次往后写，这就是一个典型的指定地址写的时序流程。

然后我们从头来看一下，首先初始化之后，总线默认空闲状态，STM32默认是从模式。为了产生一个起始条件，STM32需要写入控制寄存器CR1中的Start位，在这一位写1就可以产生起始条件了。之后STM32由从模式转为主模式，也就是多主机模型下，STM32有数据要发就要跳出来这个意思。然后控制完硬件电路之后，我们就要检查标志位来看看硬件有没有达到我们想要的状态。在起始条件之后，会发生EV5事件，这个EV5事件也可以把它当成是标志位。在STM32的手册里，都是用EVx事件来代替标志位的。这主要因为有的状态会同时产生多个标志位，所以这个EVx事件就是组合了多个标志位的一个大标志位。在库函数中，也有对应的检查EVx事件是否发生的函数。所以这里可以把它当成是一个大标志

位来理解就行。EV5事件就是SB（Start Bit）标志位为1。SB是状态寄存器的一个位，表示它硬件的状态。它存在于状态寄存器SR1中，这位置1代表起始条件已发送，这个状态寄存器是不需要手动清除的。当检测到起始条件已发送时，就可以发送一个字节的从机地址了。从机地址需要写到数据寄存器DR中，写入DR之后，硬件电路就会自动把这一字节转到移位寄存器中，再把这一个字节发送到I2C总线上，之后硬件会自动接收应答位并判断。如果没有应答，硬件就会自应答失败的标志位，然后这个标志位可以申请中断来做出提醒。在寻址完成之后，会发生EV6事件。EV6事件就是ADDR标志位为1，ADDR标志位在主模式状态下就代表地址发送结束。EV 6事件结束后，是EV8_1事件。EV8_1事件就是TxE标志位=1，移位寄存器空，数据寄存器空。这时需要我们写入数据寄存器DR进行数据发送了，一旦写入DR之后。因为移位寄存器也是空的，所以DR会立刻转到移位寄存器进行发送，这时就是EV8事件。移位寄存器非空，数据寄存器空，这时就是移位寄存器正在发数据的状态。所以流程这里，数据1的时序就产生了。这个数据寄存器和移位寄存器的配合需要把前面I2C的基本结构记好，就是发送的时候，数据先写入数据寄存器，如果移位寄存器没有数据再转到移位寄存器进行发送，这个流程要理解清楚，否则移位寄存器空，数据寄存器空就不好理解了。写入Dr将清除EV8事件，这里写入下一个数据，也就是后面的数据2，就被写入到数据寄存器里等着了，然后接收应答位之后，数据2就转入移位寄存器进行发送。此时的状态是移位寄存器非空，数据寄存器空，所以这时EV8事件就又发生了。此时数据2还正在移位发送，但下一个数据已经被写到数据寄存器等着了。按照这个流程如此重复，一旦检测到EV8事件，就可以写入下一个数据了。最后当想要发送的数据写完之后，没有新的数据可以写入到数据寄存器了，当移位寄存器当前的数据移位完成时，就进入到移位寄存器空，数据寄存器也空的状态，这个事件就是EV8_2。EV8_2是TxE=1，也就是数据寄存器空，BTF（Byte Transfer Finished）这个是字节发送结束标志位，BTF标志为置1，就是当前的移位寄存器已经移完了，该找数据寄存器要下一个数据了，但是数据寄存器没有数据。说明主机不想发了，代表字节发送结束是时候停止了。所以当检测到EV8_2时，就可以产生终止条件了，产生中止条件需要在控制寄存器CR1中将Stop位置1，这样就会在当前字节传输或当前其实条件发出后产生停止条件，到这里一个完整的时序就发送完成了。这整个过程看上去比较复杂，操作和事件都比较多，但是简单来说就是写入控制寄存器CR或者数据寄存器DR，就可以控制时序单元的发生。比如产生起始条件发送一个字节数据，时序单元发生后，检查相应的EV事件，其实就是检查状态寄存器SR来等待时序单元发送完成，然后依次按照这个流程，操作、等待、操作、等待……这样就能实现时序了。

5、主机接收

图246　主接收器传送序列图



说明：S=Start(起始条件)，S_r=重复的起始条件，P=Stop(停止条件)，A=响应，NA=非响应，EVx=事件(ITEVFEN=1时产生中断)

EV5：SB=1，读SR1然后将地址写入DR寄存器将清除该事件。
EV6：ADDR=1，读SR1然后读SR2将清除该事件。在10位主接收模式下，该事件后应设置CR2的START=1。
EV6_1：没有对应的事件标志，只适于接收1个字节的情况。恰好在EV6之后(即清除了ADDR之后)，要清除响应和停止条件的产生位。
EV7：RxNE=1，读DR寄存器清除该事件。
EV7_1：RxNE=1，读DR寄存器清除该事件。设置ACK=0和STOP请求。
EV9：ADDR10=1，读SR1然后写入DR寄存器将清除该事件。

再来继续看主机接收的流程。有7位主接收和10位主接收，其中7位主接收的时序流程是起始，从机地址+读，接收应答，接收数据，发送应答，接收数据，发送应答，最后一个数据给非应答，之后终止。可以看出这个时序是当前地址读的时序，指定地址读的复合格式。下面10位地址的当前地址读，就更复杂一些了。首先是起始发送帧头，这个帧头里的读写位是写，之后继续发送第二个字节的8位地址，这样才

能进行寻址，然后要想转入读的时序，必须再发送重复起始条件，发送帧头，这时帧头的读写位就是读的了，因为发送读的指令之后，必须要立刻转入读的时序，所以这第二个字节的地址就没有了，直接转入接收数据的时序，这就是10位地址的操作流程。接下来与发送的情况一样，主要还是看7位地址的读。

首先写入控制寄存器的Start位，产生起始条件。然后等待EV5事件，EV5事件就代表起始条件已发送，之后是寻址，接收应答。结束后产生EV6事件，EV6事件代表寻址已完成。之后数据1通过移位寄存器进行输入，EV6_1事件没有对应的事件标志，只适于接收一个字节的情况。EV6_1时数据1其实还正在移位，还没有收到，所以这个事件没有标志位。之后当这个时序单元完成时，硬件会自动根据我们的配置，把应答位发送出去。关于这个应答位的配置，在控制寄存器CR1中有一位ACK应答使能，如果写1，在接收一个字节后就返回一个应答；写0，就是不给应答。当这个时序单元结束后，就说明移位寄存器已经成功移入一个字节的数据1了，这时移入的一个字节就整体转移到数据寄存器，同时置RxNE标志位，表示数据寄存器非空，也就是收到了一个字节的数据，这个状态就是EV7事件。EV7事件就是RxNE=1，数据寄存器非空，读DR寄存器清除该事件。也就是收到数据了，当把这个数据读走之后，这个事件就没有了。EV7事件没有了，说明此时数据1被读走。当然数据1还没读走的时候，数据2就可以直接移入移位寄存器，之后数据2移位完成，收到数据2，产生EV7事件，读走数据2，EV7事件没有了，然后按照这个流程就可以一直接收数据了，最后当不需要继续接收时，需要在最后一个时序单元发生时，提前把前述的应答位控制寄存器ACK置0，并且设置终止条件请求，这就是EV7_1事件。EV7_1事件就是RxNE=1，设置ACK=0和Stop请求，也就是说明想要结束了。之后在这个时序完成后，由于设置了ACK=0，所以这里就会给出非应答，最后由于设置Stop位，所以会产生终止条件，这样接收一个字节的时序就完成了。整体来看流程和发送时差不多，写入控制寄存器CR和读取数据寄存DR，产生时序单元，然后等待相应的事件来确保时序单元完成，就是这个主机接收的时序流程。

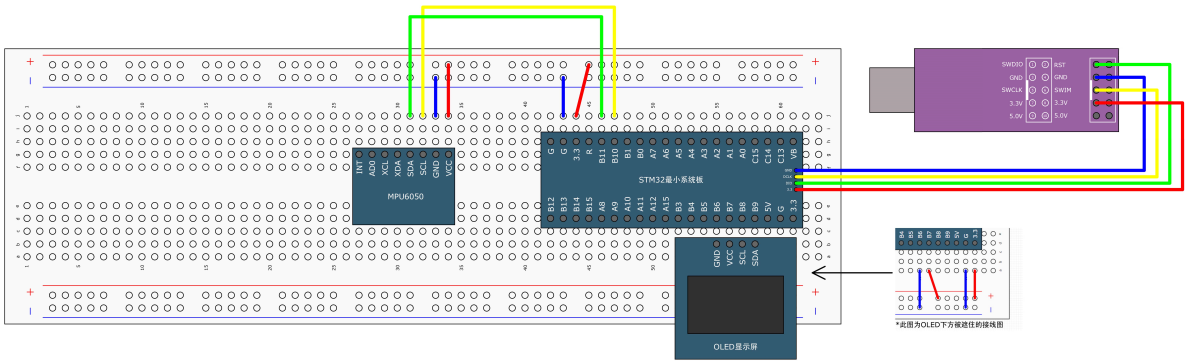
四、I2C实验

1、软件I2C读取MPU6050

（1）实验目的

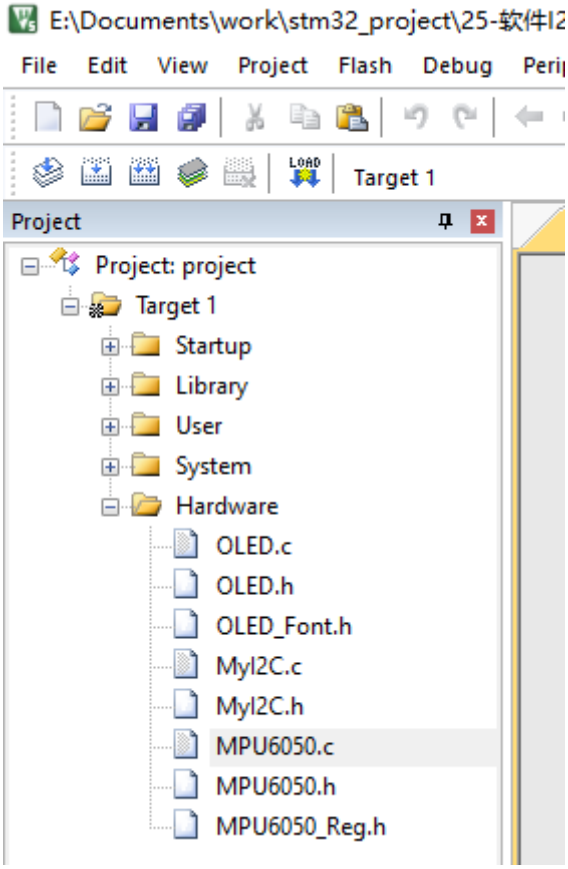
通过控制STM32利用软件实现I2C协议，对MPU6050的内部寄存器进行读写，并在OLED上显示读出的数据值。

（2）硬件电路接线图



（3）程序设计

工程结构如下图：



其中Hardware组与之前工程不一样，这个组主要增加了MyI2C、MPU6050的相关驱动代码。注意I2C的基本操作步骤：

- 首先进行I2C初始化，
- 然后读取外设ID，
- 最后读写寄存器。

具体代码展示如下：

MyI2C.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"

void MyI2C_W_SCL(uint8_t BitValue) {
    GPIO_WriteBit(GPIOB, GPIO_Pin_10, (BitAction)BitValue);     // 根据BitValue，设置SCL引脚的电平
    Delay_us(10);   // 延时10us，防止时序频率超过要求
}

void MyI2C_W_SDA(uint8_t BitValue) {
    GPIO_WriteBit(GPIOB, GPIO_Pin_11, (BitAction)BitValue);     // 根据BitValue，设置SDA引脚的电平
                                                                // BitValue要实现非0即1的特性
    Delay_us(10);   // 延时10us，防止时序频率超过要求
}

uint8_t MyI2C_R_SDA(void) {
    uint8_t BitValue;
```

```
    BitValue = GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_11);        // 读取SDA电平
    Delay_us(10);   // 延时10us，防止时序频率超过要求
    return BitValue;
}

void MyI2C_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);   // 打开GPIOB的时钟

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_OD;    // 配置开漏输出
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10 | GPIO_Pin_11;
    GPIO_Init(GPIOB, &GPIO_InitStructure);      // 初始化GPIOB

    GPIO_SetBits(GPIOB, GPIO_Pin_10 | GPIO_Pin_11);     // 设置PB10和PB11引脚初始化后默认为
高电平，释放总线状态
}

void MyI2C_Start(void) {
    MyI2C_W_SDA(1);     // 释放SDA，确保SDA为高电平
    MyI2C_W_SCL(1);     // 释放SCL，确保SCL为高电平
    MyI2C_W_SDA(0);     // 在SCL高电平期间，拉低SDA，产生起始信号
    MyI2C_W_SCL(0);     // 起始后把SCL也拉低，即为了占用总线，也为了方便总线时序的拼接
}

void MyI2C_Stop(void) {
    MyI2C_W_SDA(0);     // 拉低SDA，确保SDA为低电平
    MyI2C_W_SCL(1);     // 释放SCL，使SCL呈现高电平
    MyI2C_W_SDA(1);     // 在SCL高电平期间，释放SDA，产生终止信号
}

void MyI2C_SendByte(uint8_t Byte) {
    uint8_t i;
    for (i = 0; i < 8; i++) {
        MyI2C_W_SDA(Byte & (0x80 >> i));    // 使用掩码的方式取出Byte的指定一位数据并写入到
SDA线
        MyI2C_W_SCL(1);     // 释放SCL，从机在SCL高电平期间读取SDA
        MyI2C_W_SCL(0);     // 拉低SCL，主机开始发送下一位数据
    }
}

uint8_t MyI2C_ReceiveByte(void) {
    uint8_t i, Byte = 0x00;
    MyI2C_W_SDA(1);     // 接收前，主机先确保释放SDA，避免干扰从机的数据发送
    for (i = 0; i < 8; i++) {
        MyI2C_W_SCL(1);     // 释放SCL，主机机在SCL高电平期间读取SDA
        if (MyI2C_R_SDA() == 1) {
            Byte |= (0x80 >> i);    // 读取SDA数据，并存储到Byte变量
        }
        MyI2C_W_SCL(0);     // 拉低SCL，从机在SCL低电平期间写入SDA
    }
```

```
    return Byte;
}

void MyI2C_SendAck(uint8_t AckBit) {
    MyI2C_W_SDA(AckBit);    // 主机把应答位数据放到SDA线
    MyI2C_W_SCL(1);     // 释放SCL，从机在SCL高电平期间，读取应答位
    MyI2C_W_SCL(0);     // 拉低SCL，开始下一个时序模块
}

uint8_t MyI2C_ReceiveAck(void) {
    uint8_t AckBit;
    MyI2C_W_SDA(1);     // 接收前，主机先确保释放SDA，避免干扰从机的数据发送
    MyI2C_W_SCL(1);     // 释放SCL，主机机在SCL高电平期间读取SDA
    AckBit = MyI2C_R_SDA(); // 将应答位存储到变量里
    MyI2C_W_SCL(0);     // 拉低SCL，开始下一个时序模块
    return AckBit;
}
```

MyI2C.h

```
#ifndef __MYI2C_H
#define __MYI2C_H

void MyI2C_Init(void);
void MyI2C_Start(void);
void MyI2C_Stop(void);
void MyI2C_SendByte(uint8_t Byte);
uint8_t MyI2C_ReceiveByte(void);
void MyI2C_SendAck(uint8_t AckBit);
uint8_t MyI2C_ReceiveAck(void);

#endif
```

MPU6050.c

```
#include "stm32f10x.h"                  // Device header
#include "MyI2C.h"
#include "MPU6050_Reg.h"

#define MPU6050_ADDRESS 0xD0        // MPU6050的I2C从机地址

void MPU6050_WriteReg(uint8_t RegAddress, uint8_t Data) {
    MyI2C_Start();      // I2C起始
    MyI2C_SendByte(MPU6050_ADDRESS);    // 发送从机地址，读写位为0，表示即将写入
    MyI2C_ReceiveAck();     //接收应答
    MyI2C_SendByte(RegAddress);     // 发送寄存器地址
    MyI2C_ReceiveAck();     // 接收应答
    MyI2C_SendByte(Data);   // 发送要写入寄存器的数据
    MyI2C_ReceiveAck();     // 接收应答
```

```
    MyI2C_Stop();       // I2C终止
}

uint8_t MPU6050_ReadReg(uint8_t RegAddress) {
    uint8_t Data;

    MyI2C_Start();      // I2C起始
    MyI2C_SendByte(MPU6050_ADDRESS);    // 发送从机地址，读写位为0，表示即将写入
    MyI2C_ReceiveAck();     // 接收应答
    MyI2C_SendByte(RegAddress);     // 发送寄存器地址
    MyI2C_ReceiveAck();     // 接收应答

    MyI2C_Start();      // I2C重复起始
    MyI2C_SendByte(MPU6050_ADDRESS | 0x01);     // 发送从机地址，读写位为1，表示即将读取
    MyI2C_ReceiveAck();     // 接收应答
    Data = MyI2C_ReceiveByte();     // 接收指定寄存器的数据
    MyI2C_SendAck(1);       // 发送应答，给从机非应答，终止从机的数据输出
    MyI2C_Stop();       // I2C终止

    return Data;
}

void MPU6050_Init(void) {
    // 初始化I2C
    MyI2C_Init();
    MPU6050_WriteReg(MPU6050_PWR_MGMT_1, 0x01);     // 解除睡眠，选择陀螺仪时钟
    MPU6050_WriteReg(MPU6050_PWR_MGMT_2, 0x00);     // 六个轴均不待机
    MPU6050_WriteReg(MPU6050_SMPLRT_DIV, 0x09);     // 采样分频设置为10分频
    MPU6050_WriteReg(MPU6050_CONFIG, 0x06);     // 滤波参数选最大
    MPU6050_WriteReg(MPU6050_GYRO_CONFIG, 0x18);        // 陀螺仪选最大量程
    MPU6050_WriteReg(MPU6050_ACCEL_CONFIG, 0x18);       // 加速度计选最大量程
}

uint8_t MPU6050_GetID(void) {
    return MPU6050_ReadReg(MPU6050_WHO_AM_I);   // 返回WHO_AM_I寄存器的值
}

void MPU6050_GetData(int16_t *AccX, int16_t *AccY, int16_t *AccZ,
                     int16_t *GyroX, int16_t *GyroY, int16_t *GyroZ) {
    uint8_t DataH, DataL;
    // 读取加速度计三轴的数据，每个数据按高8位和低8位分别读取
    DataH = MPU6050_ReadReg(MPU6050_ACCEL_XOUT_H);
    DataL = MPU6050_ReadReg(MPU6050_ACCEL_XOUT_L);
    *AccX= (DataH << 8) | DataL;

    DataH = MPU6050_ReadReg(MPU6050_ACCEL_YOUT_H);
    DataL = MPU6050_ReadReg(MPU6050_ACCEL_YOUT_L);
    *AccY= (DataH << 8) | DataL;

    DataH = MPU6050_ReadReg(MPU6050_ACCEL_ZOUT_H);
    DataL = MPU6050_ReadReg(MPU6050_ACCEL_ZOUT_L);
```

```
    *AccZ= (DataH << 8) | DataL;
    // 读取陀螺仪三轴的数据，每个数据按高8位和低8位分别读取
    DataH = MPU6050_ReadReg(MPU6050_GYRO_XOUT_H);
    DataL = MPU6050_ReadReg(MPU6050_GYRO_XOUT_L);
    *GyroX= (DataH << 8) | DataL;

    DataH = MPU6050_ReadReg(MPU6050_GYRO_YOUT_H);
    DataL = MPU6050_ReadReg(MPU6050_GYRO_YOUT_L);
    *GyroY= (DataH << 8) | DataL;

    DataH = MPU6050_ReadReg(MPU6050_GYRO_ZOUT_H);
    DataL = MPU6050_ReadReg(MPU6050_GYRO_ZOUT_L);
    *GyroZ= (DataH << 8) | DataL;
}
```

MPU6050.h

```
#ifndef __MPU6050_H
#define __MPU6050_H

void MPU6050_WriteReg(uint8_t RegAddress, uint8_t Data);
uint8_t MPU6050_ReadReg(uint8_t RegAddress);
void MPU6050_Init(void);
uint8_t MPU6050_GetID(void);
void MPU6050_GetData(int16_t *AccX, int16_t *AccY, int16_t *AccZ,
                     int16_t *GyroX, int16_t *GyroY, int16_t *GyroZ);

#endif
```

MPU6050_Reg.h

```
#ifndef __MPU6050_REG_H
#define __MPU6050_REG_H

// 自定义MPU6050寄存器地址宏定义
#define MPU6050_SMPLRT_DIV      0x19
#define MPU6050_CONFIG          0x1A
#define MPU6050_GYRO_CONFIG     0x1B
#define MPU6050_ACCEL_CONFIG    0x1C

#define MPU6050_ACCEL_XOUT_H    0x3B
#define MPU6050_ACCEL_XOUT_L    0x3C
#define MPU6050_ACCEL_YOUT_H    0x3D
#define MPU6050_ACCEL_YOUT_L    0x3E
#define MPU6050_ACCEL_ZOUT_H    0x3F
#define MPU6050_ACCEL_ZOUT_L    0x40
#define MPU6050_TEMP_OUT_H      0x41
#define MPU6050_TEMP_OUT_L      0x42
#define MPU6050_GYRO_XOUT_H     0x43
```

```
#define MPU6050_GYRO_XOUT_L      0x44
#define MPU6050_GYRO_YOUT_H      0x45
#define MPU6050_GYRO_YOUT_L      0x46
#define MPU6050_GYRO_ZOUT_H      0x47
#define MPU6050_GYRO_ZOUT_L      0x48

#define MPU6050_PWR_MGMT_1       0x6B
#define MPU6050_PWR_MGMT_2       0x6C
#define MPU6050_WHO_AM_I         0x75

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "MPU6050.h"

uint8_t ID;
int16_t AX, AY, AZ, GX, GY, GZ;

int main(void){
    // 第一步：初始化
    OLED_Init();
    MPU6050_Init();
    // 第二步：读取外设ID
    OLED_ShowString(1, 1, "ID:");
    ID = MPU6050_GetID();       // 获取MPU6050的ID号
    OLED_ShowHexNum(1, 4, ID, 2);
    // 第三步：读寄存器
    while(1){
        MPU6050_GetData(&AX, &AY, &AZ, &GX, &GY, &GZ);      // 获取MPU6050的数据
        OLED_ShowSignedNum(2, 1, AX, 5);
        OLED_ShowSignedNum(3, 1, AY, 5);
        OLED_ShowSignedNum(4, 1, AZ , 5);
        OLED_ShowSignedNum(2, 8, GX, 5);
        OLED_ShowSignedNum(3, 8, GY, 5);
        OLED_ShowSignedNum(4, 8, GZ, 5);
    }
}
```

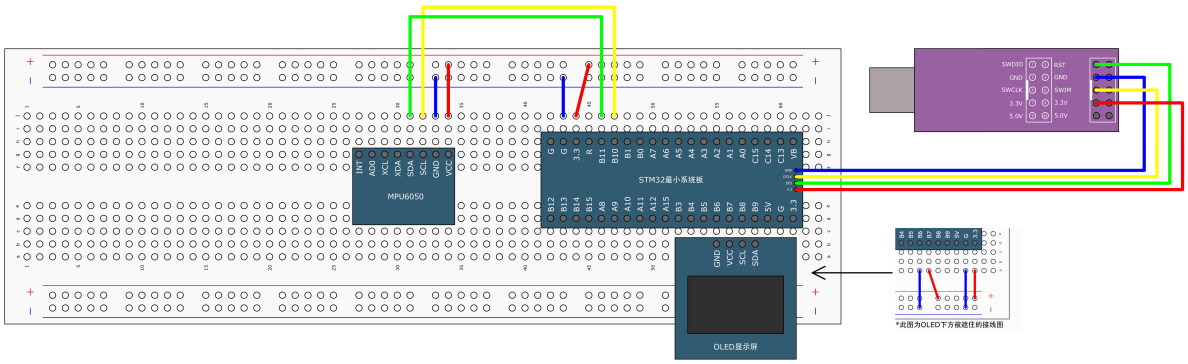
（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是MPU6050的6轴姿态数据都能够正确读取并显示到OLED上，符合预期设计。

2、硬件I2C读取MPU6050

（1）实验目的

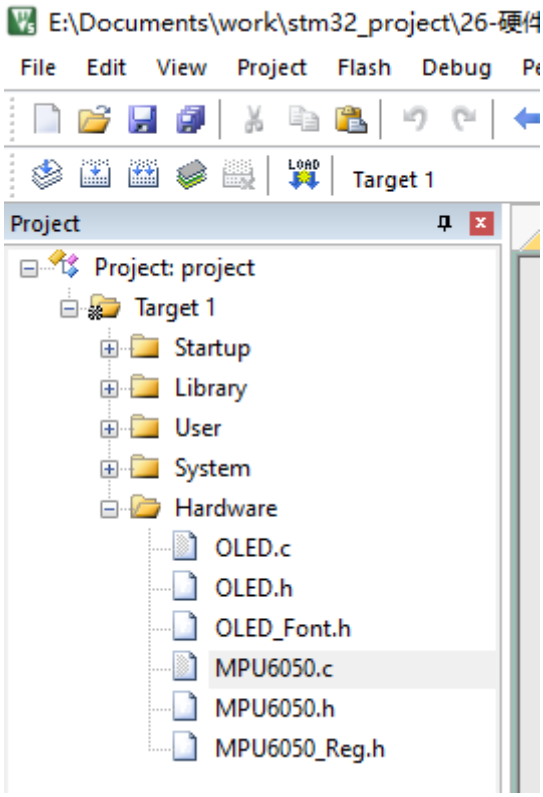
通过控制STM32利用其硬件I2C，对MPU6050的内部寄存器进行读写，并在OLED上显示读出的数据值。

（2）硬件电路接线图



（3）程序设计

工程结构如下图：



其中Hardware组与之前工程相比，这个组主要删除了自定义的MyI2C驱动，替换了MPU6050中软件实现的I2C调用的代码。注意I2C的基本操作步骤：

- 首先进行I2C初始化，
- 然后读取外设ID，
- 最后读写寄存器。

具体代码展示如下：

MPU6050.c

```
#include "stm32f10x.h"                  // Device header
#include "MPU6050_Reg.h"

#define MPU6050_ADDRESS 0xD0        // MPU6050的I2C从机地址

void MPU6050_WaitEvent(I2C_TypeDef* I2Cx, uint32_t I2C_EVENT) {
    // 为防止程序各种意外情况卡死，统一设计等待方法，超时时间到，停止等待
```

```
    uint32_t timeout;
    timeout = 10000;        // 给定超时计数时间
    // 循环等待指定事件
    while (I2C_CheckEvent(I2Cx, I2C_EVENT) != SUCCESS) {
        timeout --;
        if (timeout == 0) {
            break;      // 超时跳出等待
        }
    }
}

void MPU6050_WriteReg(uint8_t RegAddress, uint8_t Data) {
    I2C_GenerateSTART(I2C2, ENABLE);        // 硬件I2C生成起始条件
    MPU6050_WaitEvent(I2C2, I2C_EVENT_MASTER_MODE_SELECT);      // 等待EV5

    I2C_Send7bitAddress(I2C2, MPU6050_ADDRESS, I2C_Direction_Transmitter);  // 硬件I2C发送从机地址
                                                                            // 方向为发送
    MPU6050_WaitEvent(I2C2, I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED);    // 等待EV6

    I2C_SendData(I2C2, RegAddress);     // 硬件I2C发送寄存器地址
    MPU6050_WaitEvent(I2C2, I2C_EVENT_MASTER_BYTE_TRANSMITTING);    // 等待EV8

    I2C_SendData(I2C2, Data);       // 硬件I2C发送数据
    MPU6050_WaitEvent(I2C2, I2C_EVENT_MASTER_BYTE_TRANSMITTED);     // 等待EV8_2

    I2C_GenerateSTOP(I2C2, ENABLE);     // 硬件I2C生成终止条件
}

uint8_t MPU6050_ReadReg(uint8_t RegAddress) {
    uint8_t Data;

    I2C_GenerateSTART(I2C2, ENABLE);        // 硬件I2C生成起始条件
    MPU6050_WaitEvent(I2C2, I2C_EVENT_MASTER_MODE_SELECT);      // 等待EV5

    I2C_Send7bitAddress(I2C2, MPU6050_ADDRESS, I2C_Direction_Transmitter);  // 硬件I2C发送从机地址
                                                                            // 方向为发送
    MPU6050_WaitEvent(I2C2, I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED);    // 等待EV6

    I2C_SendData(I2C2, RegAddress);     // 硬件I2C发送寄存器地址
    MPU6050_WaitEvent(I2C2, I2C_EVENT_MASTER_BYTE_TRANSMITTED);     // 等待EV8_2

    I2C_GenerateSTART(I2C2, ENABLE);    // 硬件I2C生成重复起始条件
    MPU6050_WaitEvent(I2C2, I2C_EVENT_MASTER_MODE_SELECT);      // 等待EV5

    I2C_Send7bitAddress(I2C2, MPU6050_ADDRESS, I2C_Direction_Receiver); // 硬件I2C发送从机地址，方向为接收
    MPU6050_WaitEvent(I2C2, I2C_EVENT_MASTER_RECEIVER_MODE_SELECTED);   // 等待EV6

    I2C_AcknowledgeConfig(I2C2, DISABLE);       // 在接收最后一个字节之前提前将应答失能
```

```
    I2C_GenerateSTOP(I2C2, ENABLE);             // 在接收最后一个字节之前提前申请停止条件

    MPU6050_WaitEvent(I2C2, I2C_EVENT_MASTER_BYTE_RECEIVED);    // 等待EV7
    Data = I2C_ReceiveData(I2C2);       // 接收数据寄存器

    I2C_AcknowledgeConfig(I2C2, ENABLE);    // 将应答恢复为使能，为了不影响后续可能产生的读取多字节操作
    return Data;
}

void MPU6050_Init(void) {
//  MyI2C_Init();
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_I2C2, ENABLE);    // 打开I2C2时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);   // 打开GPIOB时钟

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_OD;     // 配置复用开漏输出
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10 | GPIO_Pin_11;    // 配置I2C2的PB10、PB11引脚
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    I2C_InitTypeDef I2C_InitStructure;
    I2C_InitStructure.I2C_Ack = I2C_Ack_Enable;
    I2C_InitStructure.I2C_AcknowledgedAddress = I2C_AcknowledgedAddress_7bit;
    I2C_InitStructure.I2C_ClockSpeed = 50000;       // 配置50kHz
    I2C_InitStructure.I2C_DutyCycle = I2C_DutyCycle_2;      // 低于100kHz，占空比都会是1：1
    I2C_InitStructure.I2C_Mode = I2C_Mode_I2C;
    I2C_InitStructure.I2C_OwnAddress1 = 0x00;       // 目前只用主模式，所以从模式地址随便配置一个
    I2C_Init(I2C2, &I2C_InitStructure);

    I2C_Cmd(I2C2, ENABLE);      // 使能I2C2

    MPU6050_WriteReg(MPU6050_PWR_MGMT_1, 0x01);     // 解除睡眠，选择陀螺仪时钟
    MPU6050_WriteReg(MPU6050_PWR_MGMT_2, 0x00);     // 六个轴均不待机
    MPU6050_WriteReg(MPU6050_SMPLRT_DIV, 0x09);     // 采样分频设置为10分频
    MPU6050_WriteReg(MPU6050_CONFIG, 0x06);     // 滤波参数选最大
    MPU6050_WriteReg(MPU6050_GYRO_CONFIG, 0x18);        // 陀螺仪选最大量程
    MPU6050_WriteReg(MPU6050_ACCEL_CONFIG, 0x18);       // 加速度计选最大量程
}

uint8_t MPU6050_GetID(void) {
    return MPU6050_ReadReg(MPU6050_WHO_AM_I);   // 返回WHO_AM_I寄存器的值
}

void MPU6050_GetData(int16_t *AccX, int16_t *AccY, int16_t *AccZ,
                     int16_t *GyroX, int16_t *GyroY, int16_t *GyroZ) {
    uint8_t DataH, DataL;
    // 读取加速度计三轴的数据，每个数据按高8位和低8位分别读取
    DataH = MPU6050_ReadReg(MPU6050_ACCEL_XOUT_H);
```

```
    DataL = MPU6050_ReadReg(MPU6050_ACCEL_XOUT_L);
    *AccX= (DataH << 8) | DataL;

    DataH = MPU6050_ReadReg(MPU6050_ACCEL_YOUT_H);
    DataL = MPU6050_ReadReg(MPU6050_ACCEL_YOUT_L);
    *AccY= (DataH << 8) | DataL;

    DataH = MPU6050_ReadReg(MPU6050_ACCEL_ZOUT_H);
    DataL = MPU6050_ReadReg(MPU6050_ACCEL_ZOUT_L);
    *AccZ= (DataH << 8) | DataL;
    // 读取陀螺仪三轴的数据，每个数据按高8位和低8位分别读取
    DataH = MPU6050_ReadReg(MPU6050_GYRO_XOUT_H);
    DataL = MPU6050_ReadReg(MPU6050_GYRO_XOUT_L);
    *GyroX= (DataH << 8) | DataL;

    DataH = MPU6050_ReadReg(MPU6050_GYRO_YOUT_H);
    DataL = MPU6050_ReadReg(MPU6050_GYRO_YOUT_L);
    *GyroY= (DataH << 8) | DataL;

    DataH = MPU6050_ReadReg(MPU6050_GYRO_ZOUT_H);
    DataL = MPU6050_ReadReg(MPU6050_GYRO_ZOUT_L);
    *GyroZ= (DataH << 8) | DataL;
}
```

MPU6050.h

```
#ifndef __MPU6050_H
#define __MPU6050_H

void MPU6050_WriteReg(uint8_t RegAddress, uint8_t Data);
uint8_t MPU6050_ReadReg(uint8_t RegAddress);
void MPU6050_Init(void);
uint8_t MPU6050_GetID(void);
void MPU6050_GetData(int16_t *AccX, int16_t *AccY, int16_t *AccZ,
                     int16_t *GyroX, int16_t *GyroY, int16_t *GyroZ);

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "MPU6050.h"

uint8_t ID;
int16_t AX, AY, AZ, GX, GY, GZ;

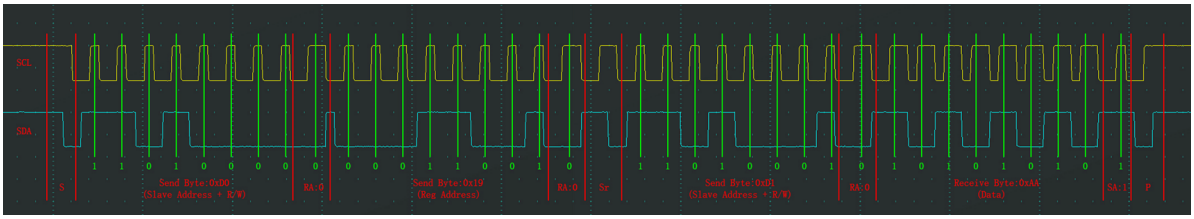
int main(void){
    // 第一步：初始化
```

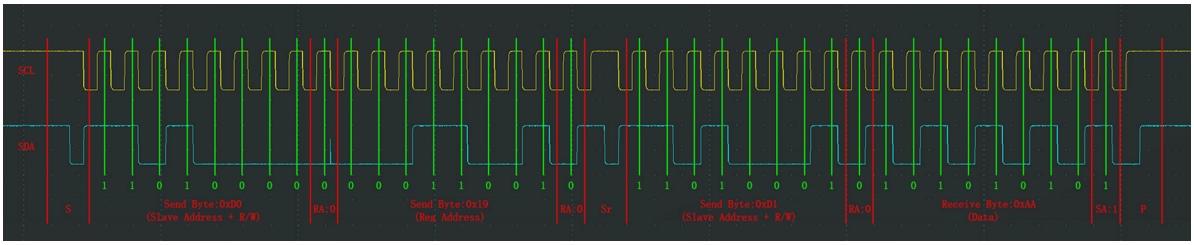
```
    OLED_Init();
    MPU6050_Init();
    // 第二步：读取外设ID
    OLED_ShowString(1, 1, "ID:");
    ID = MPU6050_GetID();       // 获取MPU6050的ID号
    OLED_ShowHexNum(1, 4, ID, 2);
    // 第三步：读寄存器
    while(1){
        MPU6050_GetData(&AX, &AY, &AZ, &GX, &GY, &GZ);      // 获取MPU6050的数据
        OLED_ShowSignedNum(2, 1, AX, 5);
        OLED_ShowSignedNum(3, 1, AY, 5);
        OLED_ShowSignedNum(4, 1, AZ , 5);
        OLED_ShowSignedNum(2, 8, GX, 5);
        OLED_ShowSignedNum(3, 8, GY, 5);
        OLED_ShowSignedNum(4, 8, GZ, 5);
    }
}
```

（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是MPU6050的6轴姿态数据都能够正确读取并显示到OLED上，符合预期设计。

五、总结

下面分别软件I2C和硬件I2C的时序波形，都反映的是指定地址读的时序。那么软件模拟的时序和硬件生成的时序有什么异同?





首先从引脚电平变化趋势上看，这两个波形都是一样的，对应的数据也都是一样的。然后从时钟线的规整程度上看，硬件I2C的波形会更加规整，硬件I2C每隔时钟的周期、占空比都非常一致。而软件I2C由于操作引脚之后都加了延时，这个延时有时候加的多，有时候加的少，所以软件时序的时钟周期占空比可能不规整。不过由于I2C是同步时序，不规整到也没有影响。此外SCL低电平写，高电平读，整个电平的任意时候都可以读写，但是一般要求保证尽早的原则，所以可以认为SCL下降沿写，上升沿读。看上面的软件I2C在下降沿之后，因为操作端口之后有些延时，所以可以看到实际等了一会才进行写入操作，后面的写入，也是等了一会。但对比来看，在硬件I2C数据写入时，都是紧贴下降沿的，SCL下降沿一到，SDA立马就切换数据了，后面也是这样的。在读的时候，读的时刻也是紧贴上升沿进行的。图中可以看到应答结束，从机在SCL下降沿立刻释放了SDA。但是软件I2C的从机过了一会儿才变换数据，所以可以看到出现了一个短暂的高电平。在硬件I2C应答结束后，SCL下降沿，从机立刻释放SDA，同时主机也立刻拉低SDA，所以能够看到出现了一个小尖峰。可以看出SDA切换都是在SCL的下降沿进行的，而这些软件操作的I2C波形就不是那么标准了。当然因为I2C同步时序的原因，这些不标准的波形也不会影响数据传输，这也是同步时序的好处。

课程小结

1. I2C通信协议是一种半双工的通信协议，它具有时钟同步和数据通信两个通道，支持一主多从和多主多从的通信方式。I2C的时序波形是本节课理解的重点，特别注意理解指定地址写，当前地址读，指定地址读三种情况。
2. MPU6050是一个6轴姿态传感器，可以用于检测无人机、平衡车的姿态与控制。其加速度计和陀螺仪的工作是需要理解的。
3. STM32的I2C基本结构原理是学习掌握的重点，配合主机发送和主机接收的时序图的学习才方便掌握如何通过程序控制外设。

随堂作业

1. 请讲解I2C指定地址写，当前地址读，指定地址读的时序。
2. 请谈谈STM32的I2C外设的实现过程。